Material didático - Conceitos fundamentais

Site: Ambiente Virtual de Aprendizagem - UFR
Course: ALGORITMOS E PROGRAMAÇÃO
Book: Material didático - Conceitos fundamentais
Printed by: Nabor do Nascimento da Silva
Date: Sunday, 5 April 2026, 4:44 PM

Table of contents

1. Apresentação

2. O processo de produção de software
2.1. Ciclo de vida de software
2.2. Saiba mais sobre

3. Software
3.1. Classificação de software

4. Linguagens de programação
4.1. Como funciona uma linguagem de programação?
4.2. Linguagem Python
4.3. Saiba mais sobre linguagens de programação

5. Compilador

6. Interpretador

7. Saiba mais: Funcionamento de compiladores e interpretadores

8. Curiosidade: como eram os computadores na década de 80

1. Apresentação

Crédito da imagem: Imagem de Freepik

Apresento a seguir conceitos de introdução à disciplina.

Bom estudo!
Prof. Waine

2. O processo de produção de software

O que é desenvolvimento de software?

O processo de construção de software é um processo distinto dos demais processos produtivos.

O processo de desenvolvimento de software é visto como um esforço de desenvolvimento único, muitas vezes com elementos de produção artesanal, com objetivos (na verdade um conjunto de funções) pré-definidos, dentro de prazos e custos controlados.

O processo de desenvolvimento de produto parecido com o desenvolvimento de software, em uma analogia bem simples, é a construção civil, por exemplo, a construção de uma casa.

Programar, nesse sentido, estaria mais próximo, analogicamente, à atividade de assentar tijolos com cimento.

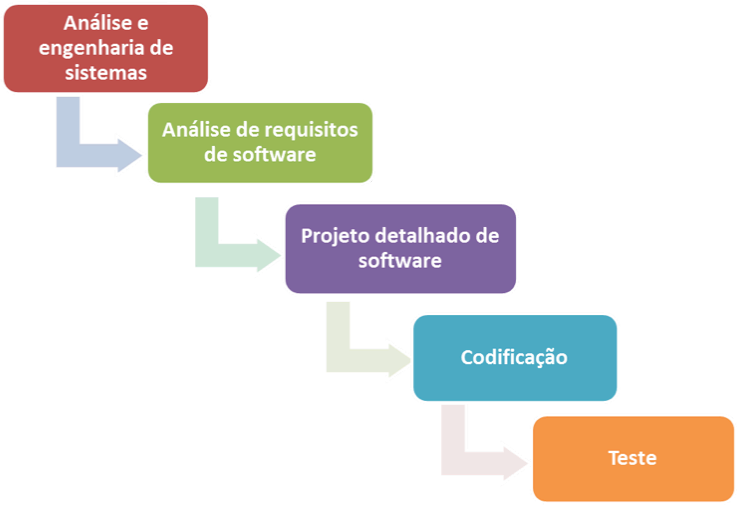
Entretanto assentar tijolos não é a primeira atividade na construção civil. A mesma coisa ocorre com o desenvolvimento de software. Desenvolver algoritmos e programá-los em uma linguagem de programação é uma das fases do desenvolvimento de software.

2.1. Ciclo de vida de software

Roger Pressman apresenta o processo tradicional de desenvolvimento de software, organizado em uma lógica de modelo que ficou conhecido como Modelo Cascata, Sequencial Linear ou ainda Ciclo de Vida Clássico.

A ideia é que há uma sequencia de fases, e que, para que a fase seguinte se inicie, a fase anterior deve ser finalizada. Veja o esquema na Figura 1 a seguir.

Figura 1 - O ciclo de vida Cascata (também chamado de clássico, linear ou sequencial)



Os objetivos gerais de cada uma das fases do ciclo de vida Cascata são apresentados a seguir.

1. **Análise e engenharia de sistemas** – envolve a coleta de requisitos preliminar, que basicamente são utilizados para estudos de planejamento de viabilidade de execução. O projeto é estudado na perspectiva da Teoria de Sistemas, como se fosse um grande sistema que pode estar contido em um outro sistema ou que pode conter subsistemas.

2. **Análise de requisitos de software** – o processo de coleta dos requisitos é intensificado e concentrado especificamente nos detalhes funcionais e não funcionais. Deve-se compreender o domínio da informação, as funções, o desempenho e as interfaces exigidas. Os requisitos são documentados e revistos com o cliente.

3. **Projeto detalhado de software** – os requisitos do software são traduzidos para um conjunto de representações na forma de algoritmos, diagramas e outras formas de representação, que podem ser avaliadas quanto à qualidade, antes que a codificação se inicie.

4. **Codificação** – as representações do projeto são codificadas em uma linguagem de programação, resultando em módulos que compõem o software final.

5. **Teste** – o software é testado e validado para que seja verificado se os requisitos definidos estão presentes nas funcionalidades que foram construídas.

6. **Manutenção** – Provavelmente o software sofrerá mudanças depois que for entregue ao cliente. As causas das mudanças: erros, adaptação do software para acomodar mudanças em seu ambiente externo e exigência do cliente para acréscimos funcionais e de desempenho. A Manutenção pode demandar a execução de qualquer uma das fases de desenvolvimento, dependendo da necessidade.

Há muito mais detalhes nesse processo e quem quiser saber mais, é só ler um bom livro de Engenharia de Software, como é o caso do livro do Roger Pressman, citado na bibliografia ao final desse material.

A disciplina de Algoritmos e programação de computadores

O desenvolvimento de algoritmos ocorre na **fase de projeto**. Nessa fase, o projetista recebe em mãos um documento chamado Documento de Requisitos, que detalha o problema a ser projetado.

Dependendo da complexidade do problema, ele pode ser quebrado em partes menores, até que uma pequena parte possa ser suficientemente compreendida para que seja algoritmizada de forma que seja possível programá-la em uma linguagem de programação.

O desenvolvimento de código ou do programa ocorre na **fase de codificação**.

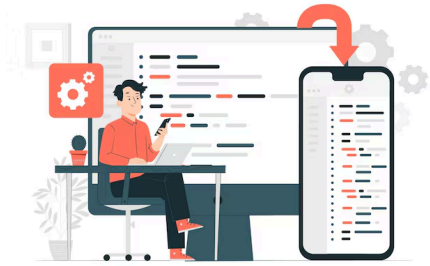
Embora durante a disciplina também façamos alguns testes em nossos programas, a atividade de teste de sofware formal é muito mais rigorosa, seguindo padrões e processos complexos.

2.2. Saiba mais sobre

Para saber mais sobre o processo de desenvolvimento de software, consulte a Norma ISO 12207.

- Link: ISO 12207

3. Software



Crédito da imagem: Imagem de storyset no Freepik

Algoritmos computacionais são implementados em linguagens de progração e são chamados de código fonte. Quando são executados são chamados de software.

Software, em sentido amplo é um conjunto de instruções que orientam um dispositivo eletrônico a realizar tarefas específicas. É como a receita de bolo para um computador: ele precisa das instruções corretas para funcionar. Pense no software como a parte invisível de um dispositivo eletrônico, a parte que diz ao hardware (a parte física) o que fazer.

Software, na Engenharia de Software é:

- "**conjunto de itens de configuração que são construídos, de alguma forma, em todas as etapas do desenvolvimento**".
- Roger S. Pressman

Os principais componentes inter-relacionados incluem:

1. **Programas (Código Fonte)**: instruções que fazem o computador realizar uma função específica.
2. **Documentos**: materiais que descrevem a operação e uso do software, essenciais para compreensão e manutenção.
3. **Dados de Configuração**: dados específicos necessários para a configuração, personalização e execução do software.

Para Pressman, esses itens de configuração são fundamentais para gerenciar e controlar as mudanças no software, uma vez que ele é continuamente modificado e atualizado ao longo de seu ciclo de vida.

3.1. Classificação de software

Podemos classificar o software em **duas** categorias principais:

- **Software de Sistema:** É o software básico que gerencia o funcionamento geral do dispositivo.
  - Exemplos: sistemas operacionais (Windows, macOS, Android), antivírus, drivers (programas que conectam o hardware ao software).
- **Software de Aplicação:** São os programas que usamos para realizar tarefas específicas.
  - Exemplos: navegadores de internet (Chrome, Firefox), editores de texto (Word, LibreOffice Writer), editores de imagem (Photoshop, GIMP), jogos eletrônicos, softwares de edição de vídeo, linguagens de programação, etc.

Software é construído com uma linguagem de programação, que é um software de aplicação.

4. Linguagens de programação

Uma **linguagem de programação** é como um idioma que permite comunicarmos com um computador e instruí-lo a realizar tarefas específicas. É como um conjunto de regras e símbolos que definem como escrever instruções para o computador entender e executar.

O conjunto de instruções para o computador executar é chamado de programa de computador ou sofware.

Um programa de computador é escrito em uma linguagem de programação.

Logo, uma linguagem de programação possui um conjunto de regras sintáticas e semânticas que devem ser estritamente seguidas para que a máquina possa entender o que deve ser feito.

Tipos de linguagens de programação

Linguagens de programação podem ser classificadas em

- baixo nível e
- alto nível.

a) Linguagem de baixo nível (ou código de máquina):

A linguagem de máquina, também conhecida como código de máquina, é a linguagem de programação de mais baixo nível que um computador pode entender diretamente.

Ela representa as instruções específicas executadas diretamente pelo processador do computador em seu formato binário, consistindo de sequências de 0s e 1s.

Cada tipo de processador tem sua própria linguagem de máquina única, e os programas escritos nessa linguagem são diretamente interpretados e executados pelo hardware do processador. Cada instrução de máquina representa uma operação específica, como adição, subtração, carregamento de dados na memória, transferência de dados entre registradores e outras tarefas de baixo nível.

Veja a seguir um exemplo em linguagem de máquina **Assembler** para mostrar na tela a mensagem "Olá mundo":

assembly Copy code

```assembly
section .data
    hello_message db "Olá mundo!", 0   ; String de mensagem
    hello_len equ $ - hello_message    ; Comprimento da string

section .text
    global _start

_start:
    ; Escreve a mensagem na saída padrão (por exemplo, console)
    mov eax, 4             ; syscall para sys_write
    mov ebx, 1             ; file descriptor 1 (stdout)
    mov ecx, hello_message ; endereço da mensagem
    mov edx, hello_len     ; comprimento da mensagem
    int 0x80               ; chama o sistema operacional

    ; Termina o programa
    mov eax, 1             ; syscall para sys_exit
    xor ebx, ebx           ; código de saída 0
    int 0x80               ; chama o sistema operacional
```

b) Linguagem de alto nível:

As linguagens de programação de alto nível são aquelas que, grosso modo, estão mais próximas da linguagem humana.

Exemplo em Linguagem Python:

print ("Olá mundo!")

Em uma linguagem de alto nível, o comando print é uma abstração que oculta os detalhes de baixo nível envolvidos na impressão de texto na tela ou em outros dispositivos de saída.

Isso significa que os programadores não precisam se preocupar com a implementação exata de como o texto é exibido no console ou em outros meios. Em vez disso, eles simplesmente usam o comando print com os argumentos adequados e confiam que a linguagem cuidará do resto.

Exemplos de linguagens de programação

Existem diversas linguagens de programação, cada uma com suas características e finalidades. Algumas das mais populares incluem:

- **Python:** Uma linguagem versátil e de fácil aprendizado, usada para diversos fins, como desenvolvimento web, análise de dados, inteligência artificial e automação.
- **Java:** Uma linguagem robusta e segura, utilizada para desenvolver aplicativos Android, softwares corporativos e sistemas embarcados.
- **JavaScript:** Uma linguagem essencial para desenvolvimento web front-end, permitindo criar interfaces interativas e dinâmicas.
- **C++:** Uma linguagem poderosa e de alto desempenho, utilizada para desenvolver jogos eletrônicos, softwares gráficos e sistemas críticos.

4.1. Como funciona uma linguagem de programação?

Para programar, escrevemos um código em uma linguagem específica, utilizando um **editor de texto** ou um **ambiente de desenvolvimento integrado (IDE).** Esse código é composto por instruções, variáveis, funções e outros elementos que definem o que o programa deve fazer.

O código ou programa, propriamente dito, é a representação padrão de um conjunto de comandos, ou seja, um algoritmo, em uma linguagem de programação que será compreendida pelo computador.

O código escrito na linguagem de programação precisa ser traduzido para que o computador possa entender.

Essa tradução é feita por meio de um **compilador** ou **interpretador**, que converte o código fonte na linguagem humana para a linguagem de máquina que o computador compreende.

4.2. Linguagem Python

O Python é uma linguagem de programação de alto nível versátil e amplamente utilizada, conhecida por sua simplicidade, legibilidade e recursos poderosos. É frequentemente recomendada para iniciantes devido à sua sintaxe clara e extensas bibliotecas, tornando-a uma excelente escolha para aprender conceitos de programação e explorar diversas aplicações.

A história do Python se inicia em dezembro de 1989, quando **Guido van Rossum**, um programador holandês apaixonado por legibilidade e simplicidade, embarcou em um projeto de hobby durante o Natal. Seu objetivo era criar uma linguagem de programação que fosse fácil de aprender, usar e entender, em contraste com as linguagens complexas da época.

Inspirado pelo programa de TV "Monty Python's Flying Circus", Guido deu à sua criação o nome **Python**. A linguagem logo ganhou popularidade entre programadores que buscavam uma alternativa mais amigável e eficiente.

Assista ao vídeo: O que é Python e porquê aprender

Alguns exemplos de onde o Python é utilizado atualmente:

1. Desenvolvimento Web:

- **Frameworks Populares:** Python é a linguagem por trás de frameworks web robustos e escaláveis como Django e Flask, utilizados para construir desde websites simples até complexas aplicações web dinâmicas.
- **Bibliotecas Versáteis:** Bibliotecas como web2py e Pyramid oferecem alternativas flexíveis para desenvolvimento web, enquanto o TurboGears facilita a criação de aplicações web rápidas e eficientes.

2. Ciência de Dados e Aprendizado de Máquina:

- **Análise e Manipulação de Dados:** O Python reina na área de ciência de dados, com bibliotecas como NumPy, pandas e scikit-learn fornecendo ferramentas poderosas para análise, manipulação e visualização de dados em grande escala.
- **Desenvolvimento de Algoritmos:** O Python é a linguagem preferida para cientistas de dados que constroem e implementam algoritmos de aprendizado de máquina, inteligência artificial e redes neurais artificiais, utilizando bibliotecas como TensorFlow e PyTorch.

3. Automação e Scripts:

- **Otimização de Tarefas:** O Python é ideal para automatizar tarefas repetitivas e complexas, economizando tempo e aumentando a produtividade. Scripts Python podem automatizar desde o envio de emails até a organização de arquivos e a criação de backups.

- **Gerenciamento de Infraestrutura:** O Python é amplamente utilizado para automatizar tarefas de gerenciamento de infraestrutura, como provisionamento de servidores, configuração de redes e monitoramento de sistemas.

4. Desenvolvimento de Software:

- **Aplicações Desktop:** O Python permite a criação de aplicativos desktop multiplataforma com interfaces gráficas amigáveis, utilizando frameworks como Tkinter e PyQt.
- **Desenvolvimento de Jogos:** O Python é utilizado para desenvolver jogos eletrônicos de diversos gêneros, desde jogos educativos até simulações complexas, com bibliotecas como PyGame e Kivy.
- **Desenvolvimento Científico:** O Python é a linguagem de escolha para muitos cientistas que desenvolvem softwares para análise de dados, modelagem científica e visualização de resultados.

4.3. Saiba mais sobre linguagens de programação

Linguagens de programação no Wikipedia

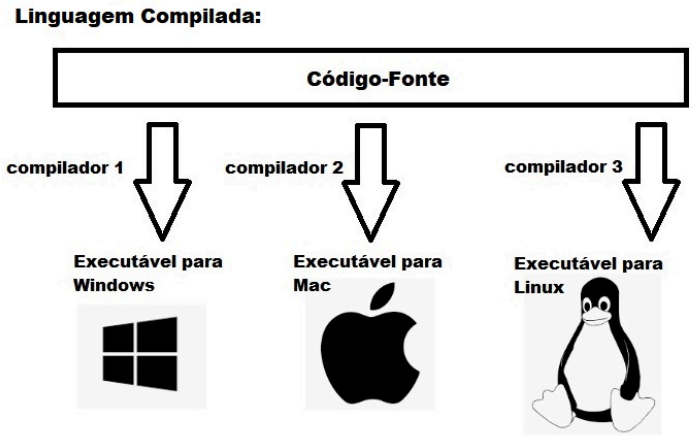
Se quiser saber mais, visite a página da Wikipedia a seguir:

- https://pt.wikipedia.org/wiki/Linguagem_de_programa%C3%A7%C3%A3o

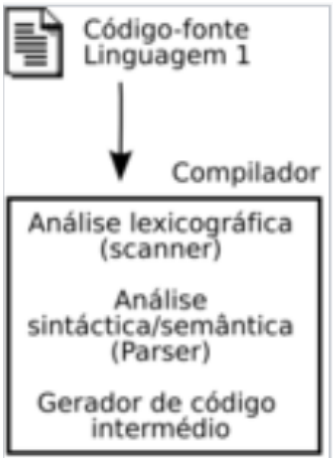
Assista ao vídeo: Portugol M00A01 Evolução das Linguagens

5. Compilador

Um **compilador** é um software traduz um programa de uma linguagem textual facilmente entendida por um ser humano (alto nível) para uma linguagem de máquina (baixo nível), específica para um processador e sistema operacional.



O compilador



O código executável (por exemplo, EXE para Windows) somente é criado depois que uma série de processos de checagens são realizados. São verificados erros lexicos e sintáticos.

Exemplos de linguagens de programação compiladas:

- C
- C++
- Pascal
- COBOL
- Delphi

Curiosidade

O primeiro compilador foi escrito por Grace Hopper, em 1952, para a linguagem de programação A-0.

Grace Murray Hopper (1906 - 1992) foi almirante e analista de sistemas da Marinha dos Estados Unidos nas décadas de 1940 e 1950, criadora da linguagem de programação de alto nível Flow-Matic - base para a criação do COBOL - e uma das primeiras programadoras do computador Harvard Mark I em 1944.

Fonte: Wikipédia

6. Interpretador

Interpretadores são programas de computador que leem um código fonte de uma linguagem de programação interpretada e o converte em código executável.

A diferença principal entre um interpretador e compilador é:

- o compilador produz um código executável, todo o programa é convertido para poder ser executado.

- o interpretador não produz código executável, o intepretador converte uma linha de cada vez e executa.

Exemplos de linguagens de programação interpretada

- BASIC
- PHP
- Python
- Forth
- JavaScript
- Ruby

Fonte: Wikipedia

7. Saiba mais: Funcionamento de compiladores e interpretadores

8. Curiosidade: como eram os computadores na década de 80

Nos anos 1980, a cultura de computadores no Brasil estava em plena expansão, impulsionada pelo aumento da disponibilidade de microcomputadores no mercado. Inicialmente, esses computadores eram vistos principalmente como ferramentas de trabalho e produtividade, utilizados em escritórios, pequenas empresas e até em alguns lares. A programação, especialmente em linguagens como BASIC, era incentivada para que os usuários pudessem explorar mais funcionalidades do computador, indo além dos usos de lazer e entretenimento, como videogames. Isso marcou o início de uma fase em que o microcomputador deixava de ser apenas uma novidade tecnológica e se tornava uma ferramenta com múltiplas utilidades práticas.

A sociedade brasileira também começou a reconhecer o potencial dos computadores como parte do desenvolvimento educacional e profissional. Muitas famílias introduziam seus filhos ao contato direto com os computadores, percebendo sua relevância para o futuro. A microcomputação emergia como uma área de conhecimento essencial, motivada pelo desejo de preparar a nova geração para as demandas tecnológicas. As empresas também se beneficiavam dessa nova tecnologia, utilizando computadores para funções administrativas e financeiras, o que demonstrava o impacto positivo e a versatilidade do microcomputador na vida cotidiana e nos negócios.

Veja a revista **Microcomputador Curso Básico** e conheça um pouco da cultura de computação nos anos 80.

Link para a revista: [Microcomputador Curso Básico]